Surprise

A language model's loss is the average surprise it feels at the real next word, and that average splits cleanly into two pieces: the entropy of language, which nothing can remove, and the model's own avoidable mistakes — which is the KL-divergence. So the loss and the KL are the same quantity up to a fixed constant, and a single line of algebra shows why two formulas that look nothing alike are the same thing.

Information: the surprise of a single outcome

A language model, at each step, is placing bets. Before it sees the next word it spreads its confidence across all the words it knows — a little probability here, a lot there. Then the real next word arrives, and we ask one question: *how badly did that catch us off guard?*

If the outcome was one we had thought likely, we are barely surprised; if we had thought it nearly impossible, we are very surprised. So we want a number — call it the **surprise**, or the *information content*, of an outcome — that depends only on the probability p we had assigned it, and we want it to behave sensibly. Three demands pin it down completely.

1. **Rarer is more surprising.** Surprise should *decrease* as p grows: a likely outcome shocks us less than an unlikely one.
2. **Certainty is no surprise.** If $p = 1$ the outcome was guaranteed, and learning it tells us nothing, so its surprise should be exactly 0.
3. **Independent surprises add.** If two *independent* things happen — the first with probability p, the second with probability q — then learning both should surprise us by the *sum* of the two separate surprises. Independent events carry separate, non-overlapping information, so their informations should simply pile up.

The third demand is the one with teeth, and it is the whole reason a logarithm appears. For independent events the probabilities *multiply*: the chance of both is $p \cdot q$. Demand 3 says the surprise of that combined event, $S(p\,q)$, must equal $S(p) + S(q)$. So the surprise function has to turn *multiplication into addition*:

$$S(p \cdot q) \;=\; S(p) + S(q).$$

There is essentially only one function that does this, and it is the logarithm — that is precisely the property $\log(pq) = \log p + \log q$. (One can see the shape forced directly: putting $q = p$ gives $S(p^2) = 2\,S(p)$, and repeating, $S(p^n) = n\,S(p)$ — surprise scales with the *exponent*, and reading off exponents is exactly what a logarithm does.) Combining this with demand 1, that surprise decreases in p, fixes the sign: we need a *minus* sign, because $\log p$ itself *increases* toward 0 as p grows to 1. The result is the definition:

$$\boxed{I(x) \;=\; -\log p(x)}$$

the **information content** (or *self-information*) of an outcome x of probability $p(x)$. Demand 2 comes for free: $I = -\log 1 = 0$ for a certain outcome. And as p slides toward 0, $-\log p$ climbs without bound — an outcome you were sure would not happen, happening anyway, is an unbounded surprise, exactly as it should feel.

Insight The logarithm is not a modelling choice; it is the *only* way to meet demand 3. Surprise lives in an additive world (informations pile up) while probability lives in a multiplicative one (independent chances multiply), and the logarithm is the unique bridge between them. Keep this in view: the *same* trick returns later in this note, when $\log$ turns the ratio p/q into the difference $\log p - \log q$ and splits the KL-divergence in two. One function, one job, used twice.

Units: bits and nats

A logarithm needs a base, and the base is just a choice of *unit* for surprise. Base 2 measures information in **bits**. A fair coin has $p = \frac{1}{2}$ for each face, so each toss carries

$$I = -\log_2 \tfrac{1}{2} = 1 \text{ bit},$$

the textbook "one bit" — the information in a single yes/no answer with no prior lean. A *biased* coin, say $p(\text{heads}) = 0.9$, is different: heads now costs only $-\log_2 0.9 \approx 0.15$ bits (barely a surprise), while the rare tails costs $-\log_2 0.1 \approx 3.32$ bits. Machine-learning work usually uses the natural logarithm instead, measuring in **nats**; the two differ only by a fixed scale, $1 \text{ nat} = 1/\ln 2 \approx 1.443$ bits. Nothing below depends on the choice, so we write just $\log$ and keep whichever unit is convenient.

Entropy: the average surprise

Information content is the surprise of *one* outcome, known after the fact. But before the outcome we can already ask: *how surprised should we expect to be?* That expected surprise, averaged over all outcomes by how often each occurs, is the **entropy** of the source:

$$H(X) = \mathbb{E}\big[I(X)\big] = -\sum_x p(x) \log p(x).$$

It is the average information per observation — the uncertainty built into X before you look.

The fair coin has $H = 1$ bit: both outcomes carry one bit, so the average is one bit, the most an un-tilted two-way choice can hold. The biased 0.9 coin has

$$H = -\big(0.9 \log_2 0.9 + 0.1 \log_2 0.1\big) \approx 0.47 \text{ bits},$$

less than one, because the coin is *predictable*: you can usually guess heads, so you are rarely surprised, and your average surprise is low. High entropy means genuinely uncertain outcomes (the fair coin); low entropy means predictable ones (the biased coin). Entropy is the average surprise you cannot talk your way out of — it belongs to the source itself.

Insight This is the object that becomes the "built-in unpredictability of language" in the sections below. When we say no model, however good, can get beneath a certain floor of surprise, that floor *is* the entropy H of the data: the average surprise that remains even when your probabilities are perfectly correct. Entropy is the irreducible part; everything

above it is the model's own fault.

A model's loss is average surprise — at the real word

Now the model. For a context, it outputs a probability $q(w)$ for every possible next word w. The true next word y arrives, and the model's surprise is its own information content for that word,

$$L \;=\; -\log q(y),$$

zero if the model was certain and right ($q(y) = 1$), large if it had given the true word little chance. This is the *same* $-\log$ of surprise as before — only now the probability inside it is the *model's* guess q, not the true probability.

Training runs the model over an enormous amount of real text and averages this surprise. Each possible word w shows up in real language some fraction of the time — call that true frequency $p(w)$ — so the average surprise weights each word's cost $-\log q(w)$ by how often it really occurs:

$$\text{loss} \;=\; \sum_w p(w)\,\big(-\log q(w)\big) \;=\; -\sum_w p(w)\,\log q(w).$$

Look closely at the two distributions in that one formula. The surprise $-\log q(w)$ is measured with the *model's* probabilities, but the averaging $\sum_w p(w)\,(\cdots)$ is done with *reality's* frequencies. That is the one difference from entropy, where *both* were the true p. This mixed quantity — real-world averaging of model-felt surprise — has a name, the **cross-entropy**, but the name matters less than the picture: *training lowers the model's average surprise at real language.*

Two parts, and only one of them can move

That average surprise is secretly *two things added together*.

Part one: unavoidable surprise. Some of it is nobody's fault. After "I flipped a fair coin and it came up ___," even a perfect predictor can only say 50/50 — it *will* be surprised one bit's worth every time, and no cleverness removes that. Language is full of this: after "I went to the ___," many words are genuinely possible. This floor is exactly the **entropy** $H(p)$ of language from the last section: the average surprise that remains even with perfect probabilities.

Part two: avoidable surprise. The extra surprise the model suffers *only because its guesses are off* — not because language is hard, but because the model is wrong. A worse model is surprised more than it needs to be. This is the part a better model shrinks.

$$\text{average surprise} \;=\; \underbrace{\text{entropy of language}}_{\text{fixed}} \;+\; \underbrace{\text{the model's avoidable mistakes}}_{\text{all that can change}}.$$

That second part — the gap between the model and reality — is the **KL-divergence**. Stripped of the name it is just "how much extra surprise the model eats purely for being wrong." It is zero only when the model's bets exactly match the true odds of language, and a positive number otherwise; the more wrong, the bigger it is.

The punchline. The entropy of language does not depend on the model at all — it is a fixed number, a property of the data. So when training drives the total down, the fixed part cannot budge, and *the only thing that can actually move is the gap*. Lowering the loss and shrinking the gap are therefore the same act.

Training a language model is dragging the model's picture of language closer to the real thing, and the KL-divergence is the tape measure for how far apart they still are.

The two formulas look nothing alike — here is the bridge

So far, so verbal. But written down, the two quantities do not look like each other, and it is worth seeing exactly why they are secretly the same.

The **loss** is the cross-entropy built in the last section but one:

$$\text{loss} \;=\; -\sum_{w} p(w)\,\log q(w),$$

a single bare logarithm, mixing the true frequencies p and the model's guesses q.

The **KL-divergence** is written with a *ratio* inside the logarithm, which is what makes it look like a different animal:

$$D_{\mathrm{KL}}(p \,\|\, q) \;=\; \sum_{w} p(w)\,\log \frac{p(w)}{q(w)}.$$

A fraction p/q where the loss had a bare q. They genuinely do not resemble one another — but they are one step apart, and the step is the very property of the logarithm that gave us surprise in the first place. There it turned a product into a sum; here it turns a quotient into a difference:

$$\log \frac{p}{q} \;=\; \log p - \log q.$$

Substitute that into the KL formula and the single sum splits into two:

$$D_{\mathrm{KL}}(p \,\|\, q) \;=\; \sum_{w} p(w)\,\big(\log p(w) - \log q(w)\big) \;=\; \underbrace{\sum_{w} p(w)\,\log p(w)}_{(\star)} \;\underbrace{-\sum_{w} p(w)\,\log q(w)}_{(\star\star)}.$$

Now read the two halves against the earlier sections.

- The second half $(\star\star)$, $-\sum_w p(w)\log q(w)$, is *exactly the loss* — letter for letter. It is the average surprise.
- The first half $(\star)$, $\sum_w p(w)\log p(w)$, uses only the true frequencies p and never mentions the model q. It is $-H(p)$: *minus the entropy* of language — the fixed floor of Section 2. Nothing the model does can change it.

So the equation reads, in the words of the last section,

$$\underbrace{D_{\mathrm{KL}}(p\|q)}_{\text{the gap}} \;=\; \underbrace{-H(p)}_{\text{a fixed constant}} \;+\; \underbrace{\text{loss}}_{\text{average surprise}}, \qquad \text{i.e.} \qquad \text{loss} \;=\; H(p) \;+\; D_{\mathrm{KL}}(p\|q).$$

The loss formula and the KL formula are not two measurements. The loss is *literally the second half of the KL*. The half that looked unfamiliar — the ratio, the extra $\log p$ — collects into one fixed number, the entropy, that the model cannot move. That the two differ by nothing but this fixed amount is the whole reason training the one is training the other — which the next section makes exact.

Insight There is a small sleight-of-hand worth savouring. To *measure* the gap you would seem to need p — the true odds of language, which nobody knows. But the only place p appears on its own is the fixed floor $(\star) = -H(p)$, and a fixed number has no effect on which way is downhill. The part that actually guides training is $(\star\star)$, the average of $-\log q$, which needs only the model's own probabilities for words that really occurred — and those you can read off directly. You descend a gap you could never compute, because its uncomputable part never moves.

Insight One asymmetry to file away. The KL above is written $D_{\mathrm{KL}}(p\|q)$, with the *true* distribution p in front — the average is over words that genuinely occur. Written the other way round, $D_{\mathrm{KL}}(q\|p)$, it is a different number; KL is a directed gap, not a symmetric distance. For ordinary training it is always the first form, because the examples are drawn from real language, so the averaging is over p whether we name it or not.

Minimising the loss is minimising the KL

Write the model's distribution as q_θ, to keep in view that it depends on the model's tunable parameters θ — its weights, the only things training is allowed to change. The decomposition from the last section becomes

$$\text{loss}(\theta) \;=\; H(p) \;+\; D_{\mathrm{KL}}(p \,\|\, q_\theta).$$

Everything now turns on a single observation: $H(p)$ **does not contain** θ**.** The entropy of language is built from the true frequencies p alone; the model never appears in it. So as training tunes θ, the term $H(p)$ sits perfectly still — a constant offset — and every bit of the θ-dependence lives in the KL term.

Same minimiser. Two functions that differ by a constant have the same shape. Picture the loss as a landscape over the space of all possible models: D_{KL} is the *same* landscape, shifted bodily upward by $H(p)$. Raising or lowering a landscape as a whole moves none of its features — the valleys stay where they are, and in particular the lowest point stays where it is. So the θ that minimises the loss is exactly the θ that minimises the KL.

Same route. It is not only the destination that agrees; so does every step toward it. Training moves downhill along the gradient, and the gradient of a constant is zero, so

$$\frac{\partial\,\text{loss}}{\partial\theta} \;=\; \underbrace{\frac{\partial\,H(p)}{\partial\theta}}_{=\,0} \;+\; \frac{\partial\,D_{\mathrm{KL}}(p\|q_\theta)}{\partial\theta} \;=\; \frac{\partial\,D_{\mathrm{KL}}(p\|q_\theta)}{\partial\theta}.$$

At every point in the landscape the two have the identical slope. Each training step is the same step whether you call it "descend the loss" or "descend the KL" — not merely the same endpoint, but the same path, nudge for nudge.

Minimising the loss *is* minimising the KL-divergence — the same minimiser, and the same gradient at every step — because the two differ only by $H(p)$, the entropy of language, which the model cannot touch.

This also says, sharply, what training is reaching for. The KL is never negative, and it is zero in exactly one situation: when $q_\theta = p$, the model's distribution equal to language's own. So the

best the model can possibly do is drive the KL to 0 — become a perfect mirror of language — and at that point the loss is not 0 but

$$\text{loss} \;=\; H(p) + 0 \;=\; H(p).$$

The entropy of language is a **floor** the loss can approach but never cross. A model is not trying to push its loss to zero — impossible, unless language itself were perfectly predictable — it is trying to push the *gap above the floor* to zero. The surprise left over at the end, $H(p)$, was never the model's to remove: it is the irreducible uncertainty of language, and reaching it means the model has learned everything there was to learn.

Insight Concretely, take the biased 0.9 coin of Section 2, whose entropy is 0.47 bits. A model learning it cannot drive its loss below 0.47 bits, however perfectly it trains — that number is the coin's own randomness, not the model's shortfall. A loss of 0.47 bits does not mean the model is still 0.47 short of the answer; it means it has the answer *exactly*, $q = p$, and the residue is the coin. So when you read a training curve, the number that matters is not how close the loss is to 0 but how close it is to $H(p)$: the distance down to that floor is the KL still left to remove.